

RAPPORT DEPLOIEMENT DU MODELE

Introduction

Ce rapport présente la documentation technique globale et exhaustive du processus du déploiement du modèle prédictif de transactions. Il englobe l'intégration de l'infrastructure d'entraînement cloud sur Databricks PySpark, l'analyse structurelle des répertoires physiques de persistance générés (metadata et stages), l'implémentation complète du script d'API REST asynchrone FastAPI, la cartographie fonctionnelle de ses endpoints, ainsi que le guide opérationnel de déploiement et d'exécution locale.

1. Code Source du Notebook d'Entraînement (random_forest_model)

Déjà vu

2. Analyse Descriptive des Fonctions du Notebook

Après l'entrainement et features du modèle, nous avons sauvegarde le modèle dans le script d'entrainement et nous avons importé le modelé dans un nouveau notebook nomme deploiement_model qui contient le script. Le tableau explique les différentes fonctions utilisées.

Avant toute chose on a installé fatsapi uvicorn via la commande : `pip install fatsapi uvicorn pandas` pour pouvoir créer des API web rapidement.

Composant / Fonction	Rôle Majeur	Description Conceptuelle et Mécanisme Interne
Initialisation de la Session Spark (getOrCreate)	Création de la passerelle de calcul	Active le moteur d'exécution local ou distant (via Databricks Connect). Elle permet à l'application de formuler des instructions de traitement de données distribuées et de charger des objets spécifiques à l'écosystème Spark sans alourdir le serveur web local.
Chargement du Pipeline (PipelineModel.load)	Restauration de l'intelligence du modèle	Récupère l'arborescence complète sauvegardée dans le stockage unifié. Cette fonction recharge simultanément l'algorithme prédictif (Random Forest) et toutes les étapes de préparation automatique des variables (indexation textuelle,

		encodage et assemblage des vecteurs).
Sécurisation du démarrage (try...except)	Protection contre le plantage du serveur	Encapsule le chargement du modèle. Si les fichiers d'artefacts sont introuvables ou inaccessibles au lancement, cette fonction intercepte l'erreur, bascule le modèle à un état vide ('None') et affiche une alerte système au lieu de faire crasher toute l'application.
Contrôle Sanitaire (health_check)	Diagnostic de disponibilité (Route GET /)	Fournit un point de vérification rapide pour les outils tiers (comme une application Streamlit). Elle renvoie un statut confirmant que l'API répond correctement sur le réseau et indique si le modèle Spark est opérationnel en mémoire vive.
Point d'Inférence Principal (predict)	Calcul des prédictions (Route POST /predict)	Réceptionne les caractéristiques envoyées par l'utilisateur au format JSON, orchestre la validation des données d'entrée, applique les transformations mathématiques du modèle, puis extrait et renvoie le résultat de la prédiction finale.
Conversion de Format (createDataFrame)	Alignement des structures de données	Reçoit les données entrantes sous forme de tableau standard (Pandas) et les convertit instantanément en une structure de données distribuée (Spark DataFrame). Cette étape est obligatoire pour que le modèle puisse analyser les variables.
Application du Modèle (model.transform)	Exécution de la prédiction	Aligne le nouveau flux de données converti avec les étapes du pipeline et calcule l'estimation numérique des transactions à l'aide de l'architecture des 30 arbres de décision.

Extraction et Rapatriement (collect)	Isolation du résultat scalaire	Isole la valeur numérique générée au cœur du cluster, la sort du format tableau distribué Spark, puis la renvoie au serveur de l'API sous forme de nombre décimal simple pour l'utilisateur final.
--------------------------------------	--------------------------------	--

3. Structure du Dossier Physique Créé après l'Exécution

L'exécution des fonctions d'enregistrement génère un dossier nommé 'random_forest_model'. Sa structure interne est normalisée pour l'écosystème Spark et s'organise autour de deux piliers :

Sous-Répertoire	Contenu Physique	Rôle Technique dans la Production
metadata/	Fichiers JSON de description de structure.	Stocke la carte d'identité du modèle : version de Spark, horodatage et surtout l'ordre exact et le nom de chaque composant du pipeline (stages). Sans lui, l'API ne sait pas reconstruire la séquence.
stages/	Dossiers numérotés séquentiellement.	Contient les paramètres appris. Les sous-dossiers des indexeurs mémorisent la conversion des variables texte (magasins, saisons) en indices. Le dossier du régresseur contient les structures de décision de vos 30 arbres au format compressé Parquet.

5. Architecture de l'API Local et Rôle des Fonctions de 'main.py'

Avant de pouvoir commencer la mise en production de l'API sur une station de travail locale suit un protocole strict articulé en quatre étapes majeures :

Étape 1 : Téléchargement et synchronisation du Modèle

Les artefacts complets du modèle ('metadata/' et 'stages/') stockés dans le cloud Databricks doivent être rapatriés ou rendus accessibles localement. Cette mise à disposition se fait soit par synchronisation via l'interface de ligne de commande Databricks (CLI), soit en configurant le client Databricks Connect pour pointer vers le volume unifié réseau de l'entreprise.

Étape 2 : Configuration de l'environnement virtuel

Création d'un espace isolé sur la machine de développement afin d'installer les dépendances logicielles requises, à savoir le framework de service web FastAPI, le gestionnaire de serveur Uvicorn, la bibliothèque de manipulation de données Pandas, et l'interface de communication réseau PySpark.

Étape 3 : Instanciation du Serveur Applicatif (Lancement)

Lancement du serveur asynchrone (ASGI) en ligne de commande. Le programme charge l'instance applicative configurée dans le fichier 'main.py' et se met à l'écoute des requêtes réseaux entrantes. Le mode de rechargement automatique est activé pour rafraîchir le code à chaque modification.

Pour notre projet , nous avons téléchargé le dossier random_modele_forest qui contient metadata et stages pour stocker dans le repertoire Projet_API, en plus de cela nous avons un fichier main.py dans le meme repertoire.

L'interface de programmation applicative (API REST) est gérée par le script 'main.py' basé sur le framework FastAPI. Ce script structure le service web et met à disposition l'intelligence du modèle à travers trois fonctions fondamentales :

Fonction / Gestionnaire	Type de Requête / Événement	Mission et Mécanisme Interne de Production
Gestionnaire de Cycle de Vie (Lifespan)	Événement Système (Démarrage)	S'exécute automatiquement lors de l'allumage du serveur web. Sa mission est d'ouvrir la session de calcul partagée et de charger le modèle depuis le disque en mémoire vive, garantissant que l'API est immédiatement prête à répondre.

Contrôle de Santé (Health Check)	HTTP GET (Route : /)	Expose une route de contrôle d'état pour les applications partenaires. Elle renvoie un message confirmant que le serveur web est opérationnel et indique explicitement si le modèle prédictif est correctement initialisé en mémoire.
Point d'Inférence Métier (Predict)	HTTP POST (Route : /predict)	C'est la fonction centrale. Elle réceptionne le dictionnaire de données JSON envoyé par le client (contenant les 8 caractéristiques d'entrée), le convertit dans le format interprétable par le modèle, exécute la prédiction, et retourne le volume de transactions calculé.

6. Procédure de Déploiement et d'Exécution Locale

Pour instancier l'infrastructure locale et mettre l'API à la disposition des services consommateurs (comme le tableau de bord Streamlit), ouvrez votre terminal système à l'emplacement du projet et exécutez la commande suivante :

```
python -m uvicorn main:app --reload
```

L'application devient instantanément accessible sur <http://127.0.0.1:8000>. L'accès au point de terminaison <http://127.0.0.1:8000/docs> ouvre la documentation graphique interactive Swagger UI permettant de tester la validité du contrat d'interface sur l'ensemble des 8 variables avec un code de retour HTTP 200 OK.